Resumen Unidad 4

Creado	@20 de enero de 2026 19:23
Etiquetas	

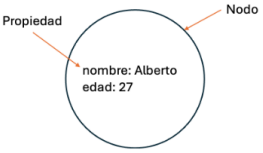
BBDD orientadas agrafos: Neo4j

Su principal caracteristica es el uso de la estrucutra de grafo para el almacenamiento de la información, estructurándolo mediante nodos que representan entidades de información, aristas que representan relaciones entre nodos además de propiedades tanto en nodos como aristas teniendo campos asociados que proporcionan información adicional.

Patrones: los grafos permiten describir patrones de una forma parecida a la que los humanos estructuran la información.

Nodo y propiedades

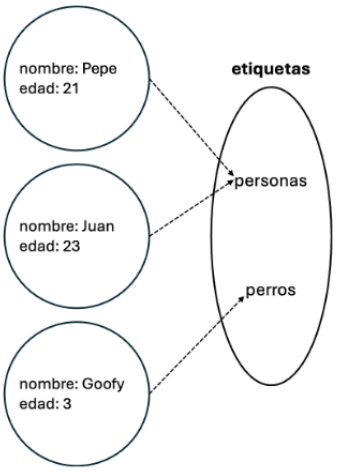
Un nodo representa una entidad de información. Los nodos tienen un conjunto de propiedades que contienen información del nodo.



Etiquetas

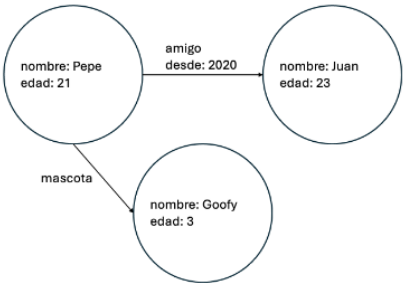
Los nodos son clasificados por etiquetas.

- Cada nodo puede tener cero o varias etiquetas.
- Las etiquetas se pueden añadir o quitar en tiempo de ejecución.
- Los nodos son agrupados en conjuntos en función de las etiquetas que tengan.
  Personas = {Pepe, Juan}
  Perros = {Goofy}
- Se pueden realizar consultas buscando por etiqueta.



Relaciones

Las relaciones son aristas entre nodos que expresan algún tipo de relación entre las entidades que enlaza. Las relaciones tienen dirección.
• Las relaciones son de un tipo.
• Las relaciones tienen propiedades que aportan más información al tipo de relación que tienen



Propiedades

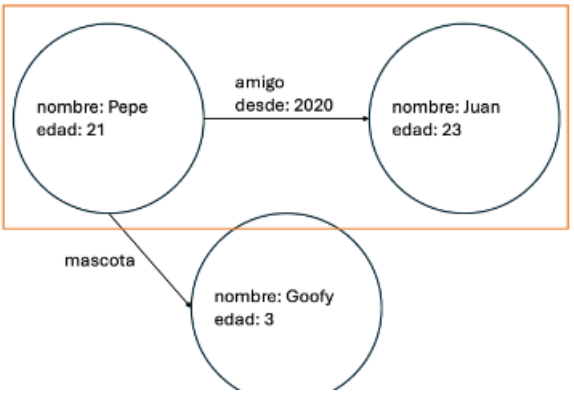
las propiedades de los nodos y relaciones son pares de clave valor.

- Las claves son cadenas de texto
- Los valores pueden ser: Números, cadenas, Booleanos, colecciones etc.

Consultas en Neo4J

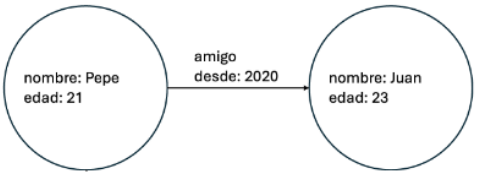
Transversal (recorrido)

Transversal es el modo en que se realizan consultas en Neo4J, se realiza un recorrido a través del grafo siguiendo las reglas especificadas. Buscar amigos de Pepe:



Path (camino)

Un path es el resultado de una consulta en Neo4J, aquellas relaciones que cumplan con las restricciones son devueltas junto con los nodos que interconecta. Amigos de Pepe:



Ejercicios de Neo4j — soluciones y comandos (completo)

Esta sección reúne definiciones breves y ejemplos prácticos (Cypher) que resuelven y amplían los puntos de las diapositivas anteriores. Está diseñada para apuntes y para copiar/pegar en `cypher-shell`.

Cada bloque tiene: resumen corto, comandos y salida esperada (cuando procede).

Nodos (CREATE, MATCH, RETURN)

Resumen: nodos con etiquetas y propiedades.

Ejemplo:

```
CREATE (p:Persona {nombre: 'Pepe', edad: 21, dni: '1234A'})
RETURN p
-- devuelve el nodo creado

MATCH (p:Persona {nombre:'Pepe'})
RETURN p.nombre, p.edad
-- "Pepe", 21
```

OPTIONAL MATCH

Resumen: incluye patrones opcionales; devuelve NULLs si no hay coincidencia.

Ejemplo práctico: devuelve persona y, si existe, su pareja

```
MATCH (p:Persona)
OPTIONAL MATCH (p)-[r:PAREJA]->(q:Persona)
RETURN p.nombre, q.nombre
-- si p no tiene relación PAREJA, q.nombre será NULL
```

WHERE, EXISTS y Regex

Ejemplo con EXISTS y regex:

```
MATCH (p:Persona)
WHERE EXISTS(p.edad) AND p.edad >= 18 AND p.nombre =~ 'Pe.*'
RETURN p
```

Ejemplo WHERE con patrón / tipo de relación:

```
MATCH (p)-[r]->(q)
WHERE type(r) =~ 'AMIG.*'
RETURN p.nombre, q.nombre, type(r)
```

MATCH + MERGE (asegurar relación)

Resumen: MERGE puede usarse para relaciones; combina búsqueda y creación.

Ejemplo — crea relación AMIGO solo si no existe:

```
MATCH (a:Persona {nombre:'Pepe'}), (b:Persona {nombre:'Juan'})
MERGE (a)-[r:AMIGO]->(b)
ON CREATE SET r.desde = 2020
RETURN r
```

WITH y agregaciones (pipeline)

Ejemplo — contar amigos por persona y ordenar:

```
MATCH (p:Persona)-[:AMIGO]->(f:Persona)
WITH p, count(f) AS num_amigos
RETURN p.nombre, num_amigos
ORDER BY num_amigos DESC
LIMIT 10
```

UNWIND y DISTINCT

Ejemplo práctico: insertar varias personas desde una lista y evitar duplicados

```
UNWIND ['Ana','Pepe','Juan','Ana'] AS nombre
MERGE (p:Persona {nombre:nombre})
RETURN collect(p.nombre)

-- Para obtener la lista única en memoria:
```

```
WITH ['Ana','Pepe','Juan','Ana'] AS lista
UNWIND lista AS n
WITH DISTINCT n
RETURN collect(n) AS unicos
```

UNION / UNION ALL

Ejemplo: combinar nombres de Personas y Animales (sin duplicados):

```
MATCH (p:Persona) RETURN p.nombre AS nombre
UNION
MATCH (a:Animal) RETURN a.nombre AS nombre
```

Para incluir duplicados usar `UNION ALL`.

Predicados: ALL / ANY / NONE / SINGLE

Ejemplos:

```
// Todas las personas del camino son adultas
MATCH path = (p:Persona)-[*2..3]-(q:Persona)
WHERE all(n IN nodes(path) WHERE n.edad >= 18)
RETURN path

// Existe al menos un vecino con propiedad x
MATCH (p:Persona)
WHERE any(n IN [(p)-[:AMIGO]->(x) | x] WHERE n.edad > 30)
RETURN p

// NONE ejemplo: ninguna relación tiene propiedad 'temporal'
MATCH (p)-[r]->()
WHERE none(x IN [r] WHERE exists(x.temporal))
RETURN p

// SINGLE: sólo un elemento cumple
WITH [1,2,3,3] AS arr
RETURN single(x IN arr WHERE x=2) AS solo_uno
```

FOREACH (actualizaciones en lote)

Ejemplo: marcar como `:Adulto` a nodos de más de 18 en un camino

```
MATCH c = (p:Persona)-[*]-(:Persona)
FOREACH (n IN nodes(c) |
    FOREACH (_ IN CASE WHEN n.edad >= 18 THEN [1] ELSE [] END | SET n:Adul
to)
)
```

ORDER BY / LIMIT / SKIP

Ejemplo: paginación simple

```
MATCH (p:Persona)
RETURN p.nombre
ORDER BY p.nombre
SKIP 10
LIMIT 5
```

SHOW INDEXES / CREATE INDEX / DROP INDEX

Ejemplos:

```
CREATE INDEX persona_nombre IF NOT EXISTS
FOR (p:Persona) ON (p.nombre)

SHOW INDEXES
-- muestra índices existentes

DROP INDEX persona_nombre
```

Nota: para texto (fulltext) en versiones anteriores se usan índices de texto:

```
CALL db.index.fulltext.createNodeIndex('idx_persona_nombre',['Persona'],
['nombre'])
```

(El comando exacto de fulltext varía por versión; comprobar la documentación del servidor.)

Índices POINT (geo) y breve nota sobre vectores

Ejemplo POINT index:

```
CREATE POINT INDEX lugar_coords IF NOT EXISTS
FOR (l:Lugar) ON (l.ubicacion)
```

Vector indexes / embeddings: Neo4j soporta índices vectoriales en versiones modernas o a través de extensiones (Graph Data Science, vector plugin). La syntaxis concreta depende de la versión; una forma posible (Neo4j 5+ con vector support) es:

```
-- Ejemplo teórico, revisar versión:
-- CREATE VECTOR INDEX idx_vec FOR (n:Item) ON (n.embedding) OPTIONS {simi
larity:'cosine'}
```

Eliminación y actualización (SET / DELETE / DETACH DELETE)

Ejemplos:

```
MATCH (p:Persona {nombre:'Pepe'})
SET p.edad = 22
```

```
MATCH (p:Persona {nombre:'Pepe'})-[r:AMIGO]->(q:Persona {nombre:'Juan'})
DELETE r

MATCH (p:Persona {nombre:'Pepe'})
DETACH DELETE p
```

Ejemplo completo — flujo: insertar, relacionar, agregar y consultar

```
UNWIND [ {nombre:'Ana', edad:30}, {nombre:'Luis', edad:25} ] AS u
MERGE (p:Persona {nombre:u.nombre})
SET p.edad = u.edad

MATCH (a:Persona {nombre:'Ana'}), (b:Persona {nombre:'Luis'})
MERGE (a)-[r:AMIGO]->(b)
ON CREATE SET r.desde = 2021

// contar amigos
MATCH (p:Persona)-[:AMIGO]->()
RETURN p.nombre, count(*) AS amigos
```